

Universidade Positivo
Padrões de Projetos
Avaliação Prática

Professor	Leandro Fabian Almeida Escobar
Turma	Turma 1
Valor	2,5 pontos (Questão 1: 1,25 ponto; Questão 2: 1,25 ponto)
Duração	1 hora

Instruções gerais

- A avaliação é individual. É permitida a consulta ao material de aula. Não é permitido o uso de ferramentas de IA generativa (chatbots, assistentes de código) nem a troca de informações com colegas.
- A entrega é feita **exclusivamente pelo Git**. Arquivos enviados por e-mail, AVA, mensageiros ou mídia removível não serão considerados.
- Crie um único repositório (GitHub ou GitLab) com o nome `padroes-projetos-av-<seu-nome>` e registre o link na entrega do BlackBoard. O repositório deve ser público.
- A entrega é separada em duas partes, uma pasta por questão, conforme a estrutura abaixo. Os commits de cada questão devem iniciar com o prefixo `q1:` ou `q2:`.

```
padroes-projetos-av-<seu-nome>/
├── README.md                (nome completo e turma)
├── questao1/
│   └── src/                 (código-fonte Java)
└── questao2/
    ├── diagrama/           (diagrama de classes: .puml, .drawio ou .png)
    └── src/                 (código-fonte Java)
```

- Serão considerados apenas os commits enviados ao repositório remoto (`git push`) até o horário de término da avaliação. A correção utiliza o último commit válido; commits posteriores serão desconsiderados.
- Cada questão vale 1,25 ponto. Os critérios de correção estão descritos ao final de cada questão.

Questão 1 (1,25 ponto): Emissão de Apólices de Seguro

Entregável: código-fonte Java na pasta `questao1/src/`. O diagrama de classes não precisa ser entregue nesta questão.

Contexto de negócio

Uma seguradora emite apólices de três linhas de produto: Auto, Residencial e Vida. Atualmente, a rotina de emissão fica em uma única classe que usa `if` e `switch` para decidir qual apólice criar, e cada nova linha de produto obriga a alterar e testar novamente esse código.

Nesse contexto, a área de tecnologia decidiu reescrever a rotina. O procedimento de emissão é o mesmo para todas as linhas: criar a apólice, calcular o prêmio mensal e imprimir o resumo. O que varia de uma linha para outra é apenas o tipo de apólice criada.

Requisitos funcionais

- **RF01 (Apólice Auto):** prêmio mensal igual a 8% do valor do veículo, dividido por 12. Documentos exigidos: CNH e CRLV.
- **RF02 (Apólice Residencial):** prêmio mensal igual a 1,5% do valor do imóvel, dividido por 12. Documentos exigidos: escritura ou contrato de locação.
- **RF03 (Apólice Vida):** prêmio mensal igual a 3% do capital segurado, dividido por 12. Documentos exigidos: documento de identidade e CPF.
- **RF04 (Resumo):** toda emissão imprime a linha de produto, o nome do segurado, o prêmio mensal e os documentos exigidos.

Requisitos não funcionais

- **RNF01:** o procedimento de emissão (criar, calcular e imprimir o resumo) deve ser escrito uma única vez, em um único lugar, e não pode ser alterado quando surgirem novas linhas de produto.
- **RNF02:** o procedimento de emissão não pode conhecer as classes concretas de apólice; ele trabalha apenas com a abstração de apólice. A decisão sobre qual apólice concreta criar fica a cargo de um componente de emissão especializado em cada linha de produto.
- **RNF03:** a inclusão de uma nova linha de produto (por exemplo, Viagem) deve exigir apenas a criação de classes novas, sem alteração de nenhuma classe existente.
- **RNF04:** o código cliente não pode instanciar diretamente nenhuma apólice concreta nem usar `if` ou `switch` para decidir o tipo de apólice.

Tarefa

Assim, identifique o padrão de projeto que atende aos requisitos não funcionais e implemente a solução em Java. A classe `Main` deve emitir uma apólice de cada linha de produto e imprimir os três resumos.

Critérios de correção

Critério	Pontuação
Escolha de um padrão de projeto criacional que atenda aos requisitos RNF01 a RNF04, aplicado com a estrutura prevista pelo catálogo GoF	0,50
Procedimento de emissão escrito uma única vez e dependente apenas da abstração da apólice (RNF01 e RNF02)	0,25
Inclusão de uma nova linha de produto possível sem alteração de classes existentes (RNF03), verificada pelo professor na correção	0,25
Código compilável e classe Main conforme a tarefa	0,25
Total	1,25

Questão 2 (1,25 ponto): Checkout Internacional de Marketplace

Entregáveis: diagrama de classes UML na pasta `questao2/diagrama/` e código-fonte Java na pasta `questao2/src/`.

Contexto de negócio

Um marketplace brasileiro passou a vender também para a Alemanha. Ao finalizar um pedido, o checkout gera três artefatos: o documento fiscal, o processamento do pagamento e a etiqueta de envio. Durante o piloto, um pedido destinado à Alemanha saiu com nota fiscal brasileira e etiqueta dos Correios, o que causou retenção na alfândega.

Pensando nisso, a equipe de compliance passou a exigir que os três artefatos de um pedido pertençam sempre ao mesmo país, e que essa garantia venha da estrutura do código.

Requisitos funcionais

- **RF01 (Brasil):** documento fiscal NF-e com ICMS de 18%; pagamento via Pix; etiqueta de envio dos Correios.
- **RF02 (Alemanha):** documento fiscal VAT invoice com imposto de 19%; pagamento via SEPA Direct Debit; etiqueta de envio da Deutsche Post.
- **RF03 (Relatório):** ao final do pedido, o checkout imprime a descrição dos três artefatos gerados.

Requisitos não funcionais

- **RNF01:** o documento fiscal, o pagamento e a etiqueta de um pedido devem ser sempre do mesmo país. O código deve impedir a combinação de artefatos de países diferentes.
- **RNF02:** a classe Checkout não pode conhecer as classes concretas de nenhum país nem usar `if` ou `switch` para decidir o país. Ela recebe, pelo construtor, um único objeto responsável por criar os três artefatos do país do pedido.
- **RNF03:** a inclusão de um novo país deve exigir apenas a criação de classes novas, sem alteração da classe Checkout.

Tarefa

Assim, identifique o padrão de projeto que atende aos requisitos não funcionais e entregue:

- a) o diagrama de classes UML da solução em draw.io (.drawio) ou imagem (.png);
- b) a implementação em Java, com uma classe `Main` que finalize um pedido para o Brasil e um pedido para a Alemanha e imprima os dois relatórios no terminal.

Critérios de correção

Critério	Pontuação
Escolha de um padrão de projeto criacional que atenda aos requisitos RNF01 a RNF03, aplicado com a estrutura prevista pelo catálogo GoF	0,40
Diagrama de classes UML completo e coerente com o código entregue	0,40
Classe Checkout dependente apenas de abstrações, sem condicionais por país (RNF02)	0,20
Código compilável e classe Main conforme a tarefa	0,25
Total	1,25